Coding & Solution

Code-Layout, Readability and Reusability:
This document evaluates the quality of the codebase in terms of its structure, readability, and reusa bility. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used standard Python indentation (4 spaces) throughout the project.
2	Proper File Structure	Files and folders are logically organized	Yes	Project follows modular architecture with separate folders for routers, services, models, frontend, and tests.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as event_description, suggestions, topics, and query clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as analyze_event(), verify_topic(), and generate_topics() use meaningful names.
5	Code Comments	Inline and block comments explain logic	Yes	Major sections and functionality are documented using comments.
6	Modular Design	Code is split into reusable functions/modules	Yes	Backend logic is divided into reusable service modules and API routers.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Reusable functions and modular architecture reduce code duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks are used in API requests and frontend operations.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Event Analyzer Service	Python (Transformers)	API routes and topic extraction	High
2	Fact Checker Service	Python (Wikipedia API)	Fact verification functionality	High

3	**History Logger**	**Python (JSON)**	**Stores and retrieves conversation history**	**High**
4	**Feedback Logger**	**Python (JSON)**	**Stores user feedback from frontend**	**High**
5	**PDF Generator**	**Python (ReportLab)**	**Generates downloadable reports**	**Medium**

Overall Code Quality Assessment:

S.No	Component / Module Name	Language / Technology
1	Event Analyzer Service	Python (Transformers)
2	Fact Checker Service	Python (Wikipedia API)
3	History Logger	Python (JSON)
4	Feedback Logger	Python (JSON)
5	PDF Generator	Python (ReportLab)

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used standard Python indentation (4 spaces) throughout the project.
2	Proper File Structure	Files and folders are logically organized	Yes	Project follows modular architecture with separate folders for routers, services, models, frontend, and tests.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as `event_description`, `suggestions`, `topics`, and `query` clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as `analyze_event()`,

				`verify_topic()`, and `generate_topics()` use meaningful names.
5	Code Comments	Inline and block comments explain logic	Yes	Major sections and functionality are documented using comments.
6	Modular Design	Code is split into reusable functions/modules	Yes	Backend logic is divided into reusable service modules and API routers.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Reusable functions and modular architecture reduce code duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks are used in API requests and frontend operations.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Event Analyzer Service	Python (Transformers)	API routes and topic extraction	High
2	Fact Checker Service	Python (Wikipedia API)	Fact verification functionality	High
3	History Logger	Python (JSON)	Stores and retrieves conversation history	High
4	Feedback Logger	Python (JSON)	Stores user feedback from frontend	High
5	PDF Generator	Python (ReportLab)	Generates downloadable reports	Medium

Overall Code Quality Assessment:

S.No	Component / Module Name	Language / Technology
1	Event Analyzer Service	Python (Transformers)
2	Fact Checker Service	Python (Wikipedia API)
3	History Logger	Python (JSON)
4	Feedback Logger	Python (JSON)
5	PDF Generator	Python (ReportLab)

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The project follows a modular architecture with separate frontend, backend, service, and testing layers. The application was successfully tested using PyTest, and all test cases passed successfully (5/5 tests passed). The project is fully functional and ready for further enhancements and deployment.